

SnapTrace

Find Yourself in Every Frame

Complete Project Documentation

AI-Powered Photo Delivery Platform

PyTorch

FastAPI

PostgreSQL

pgvector

Docker

Computer Vision · Face Recognition · Vector Search · Cloud
Deployment

2025

Table of Contents

Project Overview

Technology Stack

System Architecture

Phase 1 Dataset Preparation & Model Training

1.1 Understanding Face Quality

1.2 Dataset Sources

1.3 Dataset Structure & Labeling

1.4 Models to Fine-Tune

1.5 Training Configuration

1.6 Evaluation & Model Selection

Phase 2 FastAPI Backend Foundation

2.1 Project Structure

2.2 Database Schema

2.3 API Endpoints

2.4 Authentication & Validation

2.5 File Upload & Background Tasks

Phase 3 Computer Vision Pipeline Integration

3.1 Pipeline Components

3.2 Pipeline Flow per Image

3.3 Recognition Flow

3.4 Edge Cases & Batch Processing

Phase 4 Frontend & Complete Product

4.1 Pages & Views

4.2 Key Frontend Interactions

4.3 Download Functionality

Phase 5 Testing, Docker & Deployment

5.1 Testing Strategy

5.2 Docker Setup

5.3 Railway & Vercel Deployment

5.4 Production Checklist

Project Summary & Skills Acquired

Project Overview

SnapTrace is a full-stack, AI-powered photo delivery platform built for photographers and their clients. A photographer uploads event photos once. Clients receive a private gallery link. Anyone can find every photo they appear in simply by submitting a single selfie — no manual searching, no quality loss, no frustration.

The platform is built as a learning-first, production-grade application covering Computer Vision, PyTorch fine-tuning, FastAPI, and PostgreSQL with vector search.

The Problem

After an event, photographers share a drive link. Everyone asks to find their photos manually. Clients scroll through hundreds of images, take screenshots, and receive compressed, low-quality copies.

The Solution

Photographer uploads once. Clients open a recognition link, submit one selfie, and instantly see every photo they appear in — full resolution, ready to download.

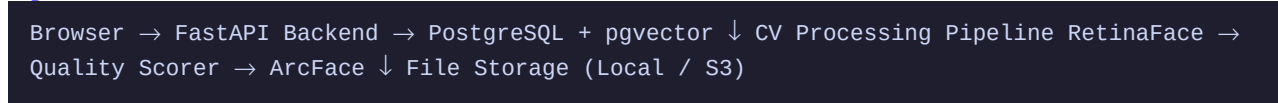
Two Access Modes

Mode	Who	Endpoint	Filter
Full Gallery	Photographer shares link	/gallery/{public_token}	None — all images visible
My Photos	Client opens recognition link	/gallery/{token}/recognize	Upload selfie → matched images only

Technology Stack

Layer	Technology	Purpose
ML Framework	PyTorch	Model training, fine-tuning, inference
Face Detection	RetinaFace (InsightFace)	Detecting and aligning faces in images
Face Recognition	ArcFace (InsightFace)	Generating 512-dim face embeddings
Quality Scorer	ResNet-18 → EfficientNet-B0	Fine-tuned binary face quality classifier
Backend	FastAPI	REST API, async processing, file handling
Database	PostgreSQL + pgvector	Metadata storage + vector similarity search
ORM	SQLAlchemy + Alembic	Database models and migrations
Task Queue	FastAPI BackgroundTasks → Celery	Async image processing pipeline
File Storage	Local → Cloudinary / AWS S3	High-quality image storage
Authentication	JWT (JSON Web Tokens)	Photographer auth + client access tokens
Frontend	HTML + CSS + Vanilla JS	Simple, clean client interface
Containerization	Docker + Docker Compose	Consistent environments
Deployment	Railway (backend) + Vercel (frontend)	Cloud deployment

System Architecture



The backend exposes REST endpoints for photographer management (auth, events, uploads) and public client access (gallery browsing, face recognition). The CV pipeline runs asynchronously as a background job, processing each uploaded image through detection → quality check → embedding → vector storage.

1

Phase 1

Dataset Preparation & Model Training

Goal

Build and label a face quality dataset, fine-tune multiple PyTorch models, evaluate all candidates, and select the best one for production use.

Learning Outcomes: PyTorch fine-tuning, transfer learning, differential learning rates, model evaluation, dataset curation, training loop design.

1.1 UNDERSTANDING FACE QUALITY

Quality	Label	Criteria
Good	1	Frontal face ($\pm 30^\circ$), both eyes visible, adequate lighting, no blur, no occlusion
Bad	0	Extreme side profile, blurry, heavily occluded, too dark, too small, multiple overlapping faces

1.2 DATASET SOURCES

Source 1 — Public Datasets

- WIDER FACE — extreme conditions, great for bad quality examples
- LFW (Labeled Faces in the Wild) — real-world faces, mixed quality
- FFHQ (Flickr-Faces-HQ) — high quality, perfect for good examples
- CelebA — varied quality, event-style photos

Source 2 — Synthetic Augmentation

Take good quality faces and apply controlled degradations: blur, rotation, darkening, partial cropping. Provides controlled bad-quality samples without manual labeling.

Target Dataset Size

Tier	Good Samples	Bad Samples	Total
Minimum	500	500	1,000
Recommended	1,500	1,500	3,000

Split: 80% train / 10% validation / 10% test

1.3 MODELS TO FINE-TUNE

Model	Params	Speed	Role in Project
ResNet-18	11M	Fast	Baseline — easiest to inspect, understand residual connections
ResNet-50	25M	Medium	Stronger baseline — depth vs width tradeoff
EfficientNet-B0	5M	Fastest	Production candidate — compound scaling concept
MobileNetV2	3M	Fastest	CPU deployment candidate — depthwise convolutions

1.4 TRAINING CONFIGURATION

Setting	Value	Rationale
Freezing Strategy	Freeze all, unfreeze last block	Preserve pretrained features, adapt final layer
Optimizer	Adam with differential LR	Backbone 1e-4, head 1e-3
LR Scheduler	CosineAnnealingLR	Smooth decay over epochs
Loss Function	BCELoss	Binary classification task
Epochs	15–20 per model	Save best checkpoint on val loss
Augmentations	Flip, ColorJitter, Rotation $\pm 15^\circ$	Training only, not validation
Normalization	ImageNet mean/std	Mandatory for pretrained models

1.5 EVALUATION & MODEL SELECTION

Metric	Why It Matters
Accuracy	Overall correctness on test set
Precision	How many flagged-good faces are actually good
Recall	How many good faces did we correctly keep
F1 Score	Balance of precision and recall — primary selection metric
Inference Time	Must be under 50ms per face crop on CPU for FastAPI
Model Size	Matters for deployment memory limits on Railway free tier

Selection Criteria: Best F1 Score with inference time under 50ms on CPU. Expected winner: EfficientNet-B0.

2

Phase 2

FastAPI Backend Foundation

Goal

Build the complete FastAPI application structure, all database models, authentication system, and all core API endpoints.

Learning Outcomes: FastAPI routing, Pydantic validation, dependency injection, JWT auth, SQLAlchemy ORM, Alembic migrations, async file handling.

2.1 DATABASE SCHEMA

users

Column	Type	Notes
id	UUID	Primary key
email	VARCHAR	Unique, indexed
hashed_password	VARCHAR	bcrypt hashed
full_name	VARCHAR	
is_active	BOOLEAN	Default true
created_at	TIMESTAMP	

events

Column	Type	Notes
id	UUID	Primary key
photographer_id	UUID	FK → users
title	VARCHAR	Event name
public_token	VARCHAR	Unique — full gallery link
recognition_token	VARCHAR	Unique — recognition link
is_processed	BOOLEAN	All images processed by CV pipeline
event_date	DATE	
created_at	TIMESTAMP	

face_embeddings

Column	Type	Notes
id	UUID	Primary key
image_id	UUID	FK → images
embedding	VECTOR(512)	pgvector column — 512-dim ArcFace embedding
face_index	INTEGER	Which face in image (0, 1, 2 ...)
quality_score	FLOAT	Score from fine-tuned quality scorer
bbox	JSONB	Bounding box coordinates

2.2 API ENDPOINTS

Authentication

Method	Endpoint	Description	Auth
POST	/auth/register	Photographer registration	No
POST	/auth/login	Returns JWT token	No
GET	/auth/me	Current user profile	JWT

Events & Images

Method	Endpoint	Description	Auth
POST	/events	Create new event	JWT
GET	/events	List photographer's events	JWT
DELETE	/events/{id}	Delete event + all images	JWT
POST	/events/{id}/tokens	Generate gallery/recognition links	JWT
POST	/events/{id}/images	Upload images (bulk)	JWT
GET	/images/{id}/download	Download original quality image	JWT

Gallery & Recognition (Public)

Method	Endpoint	Description	Auth
GET	/gallery/{token}	View full gallery	Token

Method	Endpoint	Description	Auth
GET	/gallery/{token}/images	Paginated image list	Token
POST	/gallery/{token}/recognize	Upload selfie, get matched images	Token
GET	/gallery/{token}/results/{job_id}	Poll recognition results	Token
GET	/health	API health check	No

2.3 PYDANTIC VALIDATION

Endpoint	Validation Rules
Image Upload	File type: jpg/jpeg/png only · Max 20MB per image · Max 50 images per batch
Event Creation	Event date not in the past · Title length constraints
Recognition	Exactly one face in selfie · Reject if no face · Reject if quality score too low
Token Generation	Expiry date must be future if provided · max_uses must be positive

3

Phase 3

Computer Vision Pipeline Integration

Goal

Integrate RetinaFace, the fine-tuned quality scorer, and ArcFace into a cohesive processing pipeline that runs on every uploaded image and powers the client recognition flow.

Learning Outcomes: InsightFace library, face alignment, embedding generation, batch processing, pgvector cosine similarity, error handling in ML pipelines.

3.1 PIPELINE COMPONENTS

Component	Model	Input	Output	Threshold
Face Detector	RetinaFace	Full image	Bounding boxes + landmarks	Confidence ≥ 0.85
Face Aligner	Landmark math	Crop + landmarks	Aligned 112×112 image	—
Quality Scorer	Fine-tuned ResNet/EffNet	Aligned crop	Score 0.0–1.0	Score ≥ 0.70
Embedder	ArcFace	Quality-approved crop	512-dim unit vector	—
Vector Storage	pgvector	512-dim embedding	Stored in PostgreSQL	—

3.2 PIPELINE FLOW PER IMAGE

```
Load Image → RetinaFace → For each face: [confidence check → align → quality score → if score  $\geq 0.70$ : ArcFace embed → store in pgvector] → update image.is_processed = True → check if all event images done → update event.is_processed = True
```

3.3 RECOGNITION FLOW (CLIENT)

```
Client uploads selfie → RetinaFace → exactly one face? [No → error] → Quality Scorer → acceptable? [No → "Please use better lighting"] → ArcFace → 512-dim query embedding → pgvector cosine similarity search across all event embeddings → return images where similarity  $\geq 0.45$  → sort by score descending
```

Similarity Threshold Tuning: Start at 0.45. Test with known face pairs. Too low = strangers match. Too high = person misses own photos. Expose as configurable .env value — no redeployment needed to adjust.

3.4 EDGE CASES TO HANDLE

Scenario	Handling
Image with no faces	Mark as processed, face_count = 0, no embeddings stored
Image with 20+ faces (crowd)	Process up to 30 faces, log a warning for the rest
Corrupted image file	Mark as failed, log error, continue batch without crashing
Duplicate image uploaded	Hash-based deduplication before processing
Client selfie with multiple faces	Reject immediately with clear error message
Very low resolution image	Warn but still attempt processing

4

Phase 4

Frontend & Complete Product

Goal

Build a clean, functional frontend for photographers and clients. Connect all backend features into a fully usable product.

Learning Outcomes: Connecting frontend to FastAPI, handling file uploads from browser, token-based navigation, real-time progress polling, error state design.

4.1 PAGES & VIEWS

Photographer Side (Authenticated)

Page	Purpose
/ Landing page	Product description, register/login CTA
/register	Photographer registration form
/dashboard	List of all events, quick stats
/events/new	Create new event form
/events/{id}	Image grid, upload button, processing progress, share links
/events/{id}/upload	Drag and drop bulk image upload with progress bar

Client Side (Token-based, no login)

Page	Purpose
/gallery/{token}	Full gallery — all event images, download per image
/gallery/{token}/recognize	Upload selfie, trigger recognition, view matched photos
/gallery/{token}/results	Matched photos grid with download options

4.2 KEY FRONTEND INTERACT

Bulk Upload with Progress

- Drag and drop zone accepts multiple images
- Files uploaded in batches of 10 to avoid timeout
- Progress bar shows upload progress, then processing progress
- Polls `/events/{id}/status` every 3 seconds until processing done

- Toast notification when all images are processed

Recognition Flow

- Camera capture or file upload for selfie
- Preview of uploaded selfie before submission
- Loading spinner during recognition (typically 2–5 seconds)
- Matched photos in responsive grid with similarity badge
- Download individual or all matched photos as ZIP

Error States (all must be handled gracefully)

Error	User Message
No face in selfie	Friendly message with retake instructions
Face quality too low	"Please retake in better lighting"
No matches found	Honest message — not treated as an error
Token expired	Clear message to contact photographer
Upload failure	Show which files failed, allow individual retry



Phase 5

Testing, Docker & Deployment

Goal

Write essential tests, containerize the application, and deploy to Railway (backend + database) and Vercel (frontend).

Learning Outcomes: pytest with FastAPI, Docker multi-service setup, environment management, cloud deployment, production configuration.

5.1 TESTING STRATEGY

Test Type	What Is Tested
Unit — ML	Scorer returns 0–1 float · ModelFactory raises on unknown name · Embedding is 512-dim
Integration — API	Auth flow · Event CRUD · File upload validation · Gallery token access · Recognition endpoint
End-to-End	Full photographer flow: register → upload → process → share link
End-to-End	Full client flow: open gallery → submit selfie → download matched photos

Tools: pytest + pytest-asyncio + httpx test client + pytest-cov
Target coverage: 70% minimum, focused on routes and CV pipeline.

5.2 DOCKER SETUP

Service	Image	Purpose
api	Custom Dockerfile (python:3.11-slim)	FastAPI application
db	postgres:15 with pgvector	PostgreSQL database
redis	redis:7-alpine	Task queue broker (future Celery)

Note: Use PyTorch CPU-only build (torch+cpu) to keep the Docker image under 2GB for Railway free tier. GPU is not available on Railway.

5.3 RAILWAY & VERCEL DEPL

Railway (Backend + Database)

1. Create Railway project
2. Add PostgreSQL service — Railway provides pgvector-compatible Postgres

3. Enable pgvector extension via Railway console
4. Connect GitHub repository, set environment variables
5. Configure /health endpoint as Railway health check
6. Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`
7. Run Alembic migrations as one-time job on first deploy

Vercel (Frontend)

1. Move all frontend files into /frontend directory
2. Configure vercel.json to point to frontend directory
3. Set API base URL as environment variable in Vercel dashboard
4. Update CORS in FastAPI to allow the Vercel domain
5. Deploy via Vercel CLI or GitHub integration

5.4 PRODUCTION CHECKLIST

Category	Item
Security	JWT tokens expire · CORS restricted to Vercel domain only · File size limits enforced
Performance	Database connection pooling configured · Recognition returns results under 5 seconds
Reliability	Structured JSON logs for Railway viewer · /health returns DB connection status
Quality	50-image batch completes without timeout · Downloaded images are original quality
Access Control	Gallery tokens with expiry dates enforced · Token use_count tracked correctly

Project Summary

Phase Timeline

Phase	Focus	Estimated Duration
Phase 1	Dataset preparation + PyTorch fine-tuning	1–2 weeks
Phase 2	FastAPI backend + PostgreSQL + Auth	2–3 weeks
Phase 3	CV pipeline integration + vector search	1–2 weeks
Phase 4	Frontend + complete product	1–2 weeks
Phase 5	Testing + Docker + Deployment	1 week
Total		6–10 weeks

Skills Acquired at Completion

Computer Vision	7/10	10/10
PyTorch & Fine-Tuning	4/10	8/10
FastAPI	3/10	8/10
PostgreSQL + pgvector	0/10	6/10
Docker & Deployment	3/10	6/10
Software Architecture	4/10	8/10

At the end of SnapTrace, you will have built a complete, deployed, production-grade AI application from scratch — from raw dataset curation to live cloud deployment. Every component was chosen to maximize learning depth across Computer Vision, modern backend engineering, and ML systems design.

SnapTrace — Find Yourself in Every Frame